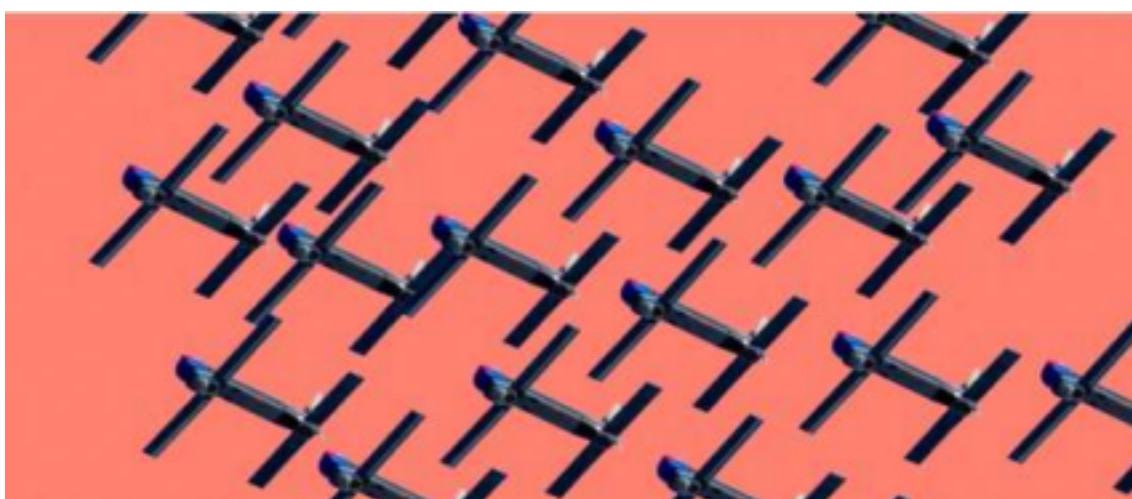


Escolha de Comandante em uma Frota de Drones

2026.1



Introdução

A guerra moderna foi radicalmente transformada pela introdução de Veículos Aéreos Não Tripulados (VANTs), popularmente conhecidos como drones, que deixaram de ser apenas ferramentas de vigilância para se tornarem atores centrais em ataques letais. A sua importância reside na capacidade de atuar em ambientes multidomínio — ar, terra e mar — com excelente relação custo-benefício e menor risco à vida dos operadores.

Neste cenário a confiabilidade dos sistemas torna-se crítica. A integridade da informação compartilhada entre os drones é fundamental, tornando algoritmos de consistência e tolerância a falhas vitais para o sucesso da missão. Em ambientes de combate reais, não ocorre apenas falha e atrasos de mensagens, mas os drones podem sofrer interferências (jamming) ou enviar dados conflitantes, um cenário classificado como "falha bizantina", onde um componente falho parece estar operando corretamente para alguns, mas não para outros.

Para resolver esses problemas, podemos utilizar algoritmos de consenso, seja para tratar falhas não bizantinas, como Paxos e Raft, ou algoritmos com tolerância a falhas bizantinas, como o BFT. Esses algoritmos são cruciais para garantir que a rede de drones concorde sobre uma decisão, mesmo se alguns membros da rede forem comprometidos ou pararem de responder, evitando assim o comportamento errático. Portanto, a evolução da guerra de drones não depende apenas da capacidade destrutiva, mas também dos softwares para garantir a resiliência e a coordenação segura em cenários incertos.

Objetivo

Uma frota de drones autônomos precisam escolher um comandante para coordenar uma missão de reconhecimento em uma área hostil. Cada drone pode propor um candidato (inclusive ele mesmo) e deve chegar a um consenso sobre quem será o comandante.

Condições do Sistema

- Os drones se comunicam por rede sem fio, sujeita a falhas de omissão e temporização.
- Um ou mais drones podem apresentar falhas bizantinas, enviando informações incorretas ou contraditórias.
- A missão exige que todos os drones corretos concordem com o mesmo comandante antes de iniciar.

Requisitos de Consenso

- Validade: Se todos os drones corretos propuserem o mesmo comandante, esse deve ser escolhido.
- Acordo: Nenhum drone correto pode escolher um comandante diferente.
- Terminação: Todos os drones corretos devem decidir em um tempo finito.
- Tolerância a falhas: O sistema deve funcionar mesmo se até 1 drone falhar de forma bizantina.

Proposta de trabalho

Escrever um software que implemente dois ou mais algoritmos de consenso, por exemplo:

1. Paxos
2. Raft
3. Bracha-Toueg
4. Mahaney-Schneider
5. Byzantine Fault Tolerance (BFT)
6. Lamport-Shostak-Pease

Descrição da Implementação

O trabalho prático consiste em escrever um programa para escolher o líder em um cenário de drones. Deverão ser formados grupos de 2 ou 3 alunos, sendo escolhido 2 ou 3 algoritmos.

Deve ser usado um ambiente de emulação como o Mininet (<http://mininet.org/>) ou

Kathará (<https://www.kathara.org/>)

Implementar os seguintes cenários:

5 drones com falhas crash, omissão e temporização (1 a 3 nós)

10 drones com falhas crash, omissão e temporização (1 a 6 nós)

5 drones com falhas bizantinas (1 a 2 nós)

10 drones com falhas bizantinas (1 a 4 nós)

Dicas

<https://libpaxos.sourceforge.net/>

<https://github.com/Tencent/phxpaxos>

<https://github.com/cocagne/paxos>

<https://github.com/el10savio/GoPaxos>

<https://raft.github.io/>

<https://ratis.apache.org/>

<https://microraft.io/>

<https://github.com/NeoResearch/libbft>

<https://github.com/rishnthan/practical-byzantine-fault-tolerance>

<https://github.com/luckydonald/pbft>

<https://github.com/varshney565/PBFT>